МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технологический университет «СТАНКИН»
(ФГБОУ ВО «МГТУ «СТАНКИН»)

Институт
информационных технологий

Кафедра
управления и информатики
в технических системах

ОТЧЕТ О ВЫПОЛНЕНИИ

ЛАБОРАТОРНОЙ РАБОТЫ ПО ДИСЦИПЛИНЕ

«Прикладное программирование**»**

по теме «Регулярные выражения. Unit-тестирование»

СТУДЕНТА *2* КУРСА бакалавриата ГРУППЫ ИДБ-25-12

(уровень профессионального образования)

Яушева Сафия Рустамовна

Направление: 09.03.03 Прикладная информатика

Отчет сдан «_____» _____2026г.

Оценка _____

Преподаватель Гетманов С. В. _____

(Ф.И.О., должность, степень, звание.) *(подпись)*

МОСКВА 2026

Оглавление

1. Постановка задачи 3

2. Описание предметной области 3

3. Регулярные выражения 4

4. Реализация программы 5

4.1. Прототипы пользовательского интерфейса 7

5. Работа с файлами и веб-страницами 8

6. Обработка исключительных ситуаций 9

7. Unit-тестирование и проверка работы 9

8. Заключение 13

Приложение. Структура проекта 14

1. Постановка задачи

Цель работы – освоение регулярных выражений в Python, получение текста из файла и веб-страницы и проверка программы unit-тестами. Для варианта 26 «Проверка и поиск чисел в двоичной системе счисления» необходимо составить регулярное выражение и написать на Python программу, которая проверяет строку, введённую пользователем, ищет двоичные числа в введённом тексте, в файле и на веб-странице по URL, а также читает входной файл и записывает найденные числа в выходной файл. К программе необходимо написать unit-тесты, обработать ошибки так, чтобы они не завершали работу программы, и оформить отчёт. Работа выполнена в системе контроля версий Git, исходный код размещён в публичном репозитории: https://github.com/schnchladp/stankin-lab/tree/main/lab2.

2. Описание предметной области

Двоичная система счисления использует две цифры: 0 и 1. В программах двоичные числа записывают последовательностью цифр, возможно со знаком и с префиксом 0b (как в Python: 0b101). Допустимая запись описана грамматикой в форме Бэкуса–Наура:

<число> ::= [<знак>] [<префикс>] <цифра> {<цифра>}

<знак> ::= "+" | "-"

<префикс> ::= "0b" | "0B"

<цифра> ::= "0" | "1"

Принятые соглашения, которые отражены в тестах: ведущие нули допустимы (0010 – число 2); пробелы и разделители разрядов (1 000, 1_000)

недопустимы; одиночный знак и одиночный префикс числом не являются. При поиске в тексте число должно быть отдельным «словом»: не частью слова (12ab), не частью более длинного числа (1012), не частью десятичной дроби (3.101), версии или даты (1.0.1, 01.01.2026). Точка или запятая в конце предложения числу не мешает.

Таблица 1. Примеры допустимых и недопустимых строк

Допустимо	**Недопустимо**	**Причина**
101	102	цифра 2
-0b1101	0b	нет цифр после префикса
+0B11	--101	два знака
0010	1 01	пробел внутри числа
0	101.5	десятичная дробь

Рассматриваются только целые двоичные числа; дробные двоичные числа и другие системы счисления не поддерживаются.

3. Регулярные выражения

В программе два выражения (файл patterns.py): первое проверяет, что вся строка является двоичным числом, второе ищет числа внутри текста.

Проверка строки целиком

```
BINARY_FULL_RE = re.compile(r"""
    ^                   # начало строки
    (?P<sign>[+-])?     # необязательный знак
    (?P<prefix>0[bB])?  # необязательный префикс 0b / 0B
    (?P<digits>[01]+)   # одна или несколько двоичных цифр
    \Z                  # конец строки
""", re.VERBOSE)
```

Поиск в тексте

```
BINARY_SEARCH_RE = re.compile(r"""
    (?<!\w)          # слева не буква, не цифра и не «_»
    (?<!\d[.,])      # слева не дробный разделитель (3.101)
    [+-]?            # необязательный знак
    (?:0[bB])?       # необязательный префикс
    [01]+            # двоичные цифры
    (?!\w)           # справа не буква, не цифра и не «_»
    (?![.,]\d)       # справа не дробный разделитель (101.5)
""", re.VERBOSE)
```

Таблица 2. Основные элементы выражений

Элемент	**Назначение**
^ и \Z	число занимает строку целиком; в отличие от $ якорь \Z не пропускает завершающий «\n», поэтому «101\n» отвергается
(?P<sign>…), (?P<digits>…)	именованные группы: из них функция binary_to_int берёт знак и цифры
[01]+	одна и более двоичных цифр (квантификатор +)
(?<!\w), (?!\w)	просмотр назад и вперёд: число не может быть частью слова, например «12ab» или «1012»
(?<!\d[.,]), (?![.,]\d)	число не может быть частью десятичной дроби: «3.101», «101.5»

Проверка выполняется методом match, поиск – методом findall, который возвращает все непересекающиеся совпадения. Утверждения просмотра не входят в найденный текст, поэтому из строки «1012» число 101 не «вырезается». Для каждого найденного числа программа вычисляет значение в десятичной системе: int(цифры, 2) со знаком.

4. Реализация программы

Программа написана на Python 3.10+; из внешних библиотек используется только requests (загрузка веб-страниц), остальное – стандартные модули re, argparse, pathlib и html.parser. Логика проекта отделена от интерфейса, ввод и вывод передаются в меню параметрами, поэтому меню можно проверять без

клавиатуры и консоли. Код оформлен по PEP 8 (строки до 79 символов), у модулей и функций есть docstring с примерами, указаны аннотации типов.

Таблица 3. Модули пакета binary_numbers

Модуль	**Назначение**
patterns.py	скомпилированные регулярные выражения
core.py	is_binary_number, find_binary_numbers, binary_to_int
sources.py	read_file, is_valid_url, fetch_page_text (requests), extract_visible_text
processing.py	process_file: входной файл → поиск → выходной файл
cli.py	консольное меню, параметры командной строки, обработка ошибок
__main__.py	запуск командой python -m binary_numbers

4.1. Прототипы пользовательского интерфейса

Макет 1. Главное меню

```
 ------------------------------------------------------------
|                        ДВОИЧНЫЕ ЧИСЛА                      |
 ------------------------------------------------------------
| 1. Проверить, является ли строка двоичным числом           |
| 2. Найти двоичные числа в введённом тексте                 |
| 3. Найти двоичные числа в файле                            |
| 4. Найти двоичные числа на веб-странице (по URL)           |
| 5. Обработать файл: входной файл -> выходной файл          |
| 0. Выход                                                   |
|                                                            |
| Выберите пункт: _                                          |
 ------------------------------------------------------------
```

Макет 2. Результат поиска в тексте

```
 ------------------------------------------------------------
|                   ПОИСК В ТЕКСТЕ (ПУНКТ 2)                 |
 ------------------------------------------------------------
| Введите текст: Числа 101, 0b11 и 102                       |
|                                                            |
| Найдено двоичных чисел: 2                                  |
|   1. 101  (в десятичной системе: 5)                        |
|   2. 0b11 (в десятичной системе: 3)                        |
 ------------------------------------------------------------
```

Макет 3. Обработка файла

```
 ------------------------------------------------------------
|                    ФАЙЛ -> ФАЙЛ (ПУНКТ 5)                  |
 ------------------------------------------------------------
| Входной файл: examples/input.txt                           |
| Выходной файл (Enter - рядом с входным): result.txt        |
|                                                            |
| Найдено двоичных чисел: 5.                                 |
| Результат записан в файл: result.txt                       |
 ------------------------------------------------------------
```

Макеты показывают структуру экранов; снимки реальной работы программы приведены в разделе 7 (рис. 1–4).

5. Работа с файлами и веб-страницами

Функция process_file(input_path, output_path) читает входной файл (UTF-8, в том числе с BOM, который ставит «Блокнот»), находит все двоичные числа и записывает результат в выходной файл. Если выходной файл не указан, он создаётся рядом с входным с именем <имя>_result.txt. Входной и выходной файл не могут совпадать, иначе исходные данные были бы потеряны. Режим доступен из меню (пункт 5) и из командной строки: python -m binary_numbers input.txt -o output.txt; код завершения 0 означает успех, 1 – ошибку.

Формат выходного файла: первая строка «Найдено: N», далее строки «число;десятичное значение».

Таблица 4. Пример входного и выходного файлов

Входной файл (examples/input.txt)	**Выходной файл (examples/output.txt)**
Лабораторная работа №2 Двоичные числа: 101, 0b1101, -1010, +0B11, 0010. Не двоичные: 102, 2024, 0x1F, 12ab, 3.101, 101.5, версия 1.0.1. Дата 01.01.2026 и число 1_000 тоже не считаются.	Найдено: 5 101;5 0b1101;13 -1010;-10 +0B11;3 0010;2

Для веб-страницы функция fetch_page_text использует requests.get с таймаутом 10 секунд; raise_for_status превращает ответы 4xx и 5xx в исключение; если сервер не указал кодировку, она определяется по содержимому, чтобы русский текст не искажался. Из HTML перед поиском убираются теги, script и style: иначе числа находились бы в атрибутах и коде страницы (например, width="100").

6. Обработка исключительных ситуаций

Каждая операция меню выполняется в блоке try/except: ошибка пользователя или внешнего источника приводит к понятному сообщению, а не к завершению программы. Завершение ввода (Ctrl+Z, Ctrl+D) и прерывание (Ctrl+C) обрабатываются корректным выходом.

Таблица 5. Обрабатываемые исключения

Исключение	**Когда возникает**	**Реакция программы**
FileNotFoundError	файла нет	«файл не найден»
UnicodeDecodeError	файл не в UTF-8	сообщение о кодировке
OSError	нет прав, путь – каталог	сообщение с причиной
OutputWriteError	не удалось записать выходной файл	сообщение с путём и причиной
ValueError	некорректный URL; выходной файл совпадает с входным	сообщение об ошибке
TypeError	в поиск передан не текст	исключение с пояснением
requests.Timeout	сервер не ответил за 10 с	«сервер не ответил»
requests.ConnectionError	нет соединения, неверный адрес	«не удалось соединиться»
requests.HTTPError	ответ 4xx или 5xx	сообщение с кодом ответа
EOFError, KeyboardInterrupt	конец ввода, Ctrl+C	«Завершение работы»

7. Unit-тестирование и проверка работы

Тесты написаны на pytest: общие данные вынесены в fixtures (conftest.py), повторяющиеся проверки оформлены через parametrize. Сетевые запросы заменены mock-объектами (unittest.mock), поэтому тесты быстрые и не зависят от интернета; для файлов используется временный каталог tmp_path. Примеры из docstring дополнительно проверены модулем doctest.

Таблица 6. Состав тестов

Файл	**Что проверяется**	**Тестов**
test_core.py	проверка строки (допустимые, недопустимые, граничные случаи), поиск в тексте, перевод в десятичную систему	52
test_sources.py	чтение файла, URL, очистка HTML, загрузка страницы с mock (кодировки, ошибка 404, таймаут, нет сети)	32
test_processing.py	файл → файл: имя результата, перезапись, BOM, ошибки чтения и записи	13
test_cli.py	все пункты меню, ветки ошибок, командная строка, Ctrl+C и EOF	26
doctest	примеры из docstring	8
Итого		123 + 8

```
$ pytest
.....................................................  [100%]
123 passed

$ pytest --doctest-modules src
8 passed
```

Покрытие кода измерено стандартным модулем trace: выполнено 100 % строк всех модулей пакета (для проверки: pytest --cov=binary_numbers --cov-report=term-missing). Тестовые данные включают «ловушки»: завершающий перевод строки («101\n»), 1012, 3.101, 12ab, «0b» без цифр, «--101», 64-разрядное число. Результаты сверены со встроенной функцией bin() Python.

Ниже приведены снимки работы программы.

```
=== Двоичные числа ===
1. Проверить, является ли строка двоичным числом
2. Найти двоичные числа в введённом тексте
3. Найти двоичные числа в файле
4. Найти двоичные числа на веб-странице (по URL)
5. Обработать файл: входной файл -> выходной файл
0. Выход
Выберите пункт: 1
Введите строку: 0b1101
«0b1101» — двоичное число (десятичное значение: 13).

=== Двоичные числа ===
1. Проверить, является ли строка двоичным числом
2. Найти двоичные числа в введённом тексте
3. Найти двоичные числа в файле
4. Найти двоичные числа на веб-странице (по URL)
5. Обработать файл: входной файл -> выходной файл
0. Выход
Выберите пункт: 1
Введите строку: 102
«102» — НЕ двоичное число

=== Двоичные числа ===
1. Проверить, является ли строка двоичным числом
2. Найти двоичные числа в введённом тексте
3. Найти двоичные числа в файле
4. Найти двоичные числа на веб-странице (по URL)
5. Обработать файл: входной файл -> выходной файл
0. Выход
Выберите пункт: 1
Введите строку: -101
«-101» — двоичное число (десятичное значение: -5).
```

Рис. 1. Проверка введённой строки (пункт 1)

```
=== Двоичные числа ===
1. Проверить, является ли строка двоичным числом
2. Найти двоичные числа в введённом тексте
3. Найти двоичные числа в файле
4. Найти двоичные числа на веб-странице (по URL)
5. Обработать файл: входной файл -> выходной файл
0. Выход
Выберите пункт: 0
До свидания!
```

Рис. 2. Проверка введённой строки (пункт 1)

```
=== Двоичные числа ===
1. Проверить, является ли строка двоичным числом
2. Найти двоичные числа в введённом тексте
3. Найти двоичные числа в файле
4. Найти двоичные числа на веб-странице (по URL)
5. Обработать файл: входной файл -> выходной файл
0. Выход
Выберите пункт: 2
Введите текст: числа 101, 0b11, 3.101, 102; 1.0.1
Найдено двоичных чисел: 2
  1. 101  (в десятичной системе: 5)
  2. 0b11  (в десятичной системе: 3)

=== Двоичные числа ===
1. Проверить, является ли строка двоичным числом
2. Найти двоичные числа в введённом тексте
3. Найти двоичные числа в файле
4. Найти двоичные числа на веб-странице (по URL)
5. Обработать файл: входной файл -> выходной файл
0. Выход
Выберите пункт: 0
До свидания!

Process finished with exit code 0
```

Рис. 3. Поиск в введённом тексте (пункт 2)

```
=== Двоичные числа ===
1. Проверить, является ли строка двоичным числом
2. Найти двоичные числа в введённом тексте
3. Найти двоичные числа в файле
4. Найти двоичные числа на веб-странице (по URL)
5. Обработать файл: входной файл -> выходной файл
0. Выход
Выберите пункт: 5
Входной файл (имя или путь): examples/input.txt
Выходной файл (Enter — результат запишется рядом с входным): examples/out_menu.txt
Найдено двоичных чисел: 5.
Результат записан в файл: examples\out_menu.txt

=== Двоичные числа ===
1. Проверить, является ли строка двоичным числом
2. Найти двоичные числа в введённом тексте
3. Найти двоичные числа в файле
4. Найти двоичные числа на веб-странице (по URL)
5. Обработать файл: входной файл -> выходной файл
0. Выход
Выберите пункт: 5
Входной файл (имя или путь): examples/none.txt
Выходной файл (Enter — результат запишется рядом с входным):
Ошибка: входной файл не найден.

=== Двоичные числа ===
1. Проверить, является ли строка двоичным числом
2. Найти двоичные числа в введённом тексте
3. Найти двоичные числа в файле
4. Найти двоичные числа на веб-странице (по URL)
5. Обработать файл: входной файл -> выходной файл
0. Выход
Выберите пункт: 0
До свидания!
```

Рис. 4. Обработка «входной файл → выходной файл» (пункт 5)

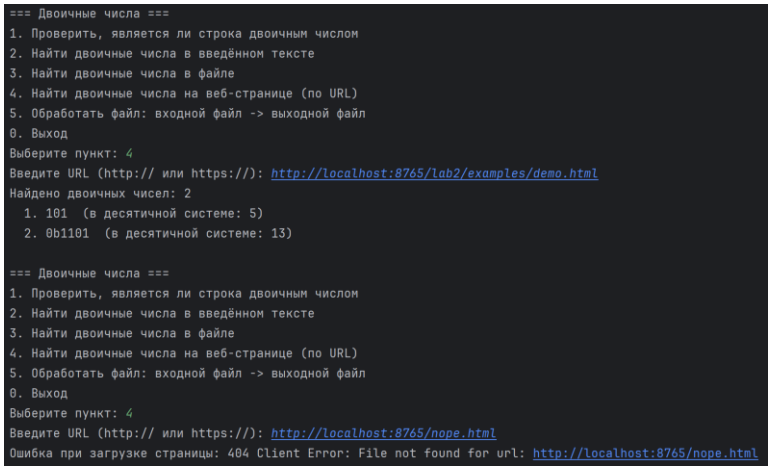


Рис. 5. Поиск на веб-странице; ошибки 404 и неверного URL (пункт 4)

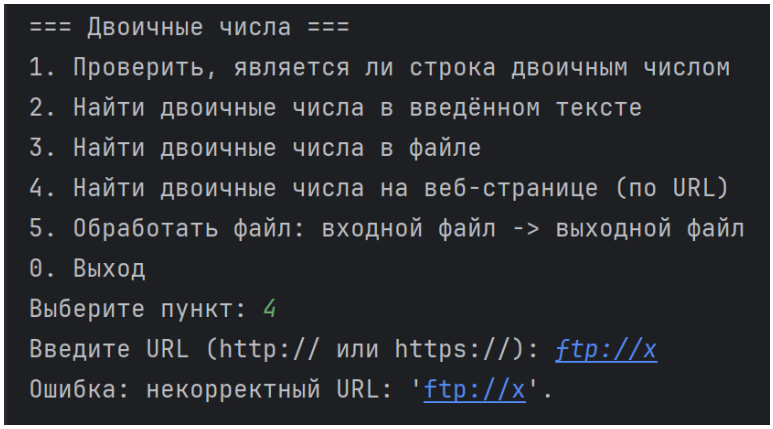


Рис. 6. Поиск на веб-странице; ошибки 404 и неверного URL (пункт 4)

8. Заключение

В ходе работы составлены и протестированы регулярные выражения для проверки и поиска двоичных чисел; реализована программа, которая работает с введённым текстом, файлом, веб-страницей и в режиме «входной файл –> выходной файл». Ошибки пользователя и внешних источников обрабатываются без аварийного завершения. Для программы написаны 123 unit-теста и 8 doctest-примеров, покрытие строк – 100 %. Получены навыки работы с модулем re, библиотекой requests, файлами, pytest, mock и Git.

Приложение. Структура проекта

src/binary_numbers/patterns.py – регулярные выражения

src/binary_numbers/core.py – проверка, поиск и перевод в десятичную систему

src/binary_numbers/sources.py – чтение файла и загрузка веб-страницы

src/binary_numbers/processing.py – обработка «входной файл → выходной файл»

src/binary_numbers/cli.py – консольное меню и параметры командной строки

src/binary_numbers/__main__.py – запуск: python -m binary_numbers

tests/ – unit-тесты pytest (conftest.py, test_core.py, test_sources.py, test_processing.py, test_cli.py)

examples/ – input.txt, output.txt (пример результата), demo.html (страница для проверки)

docs/ – отчёт (.docx и .pdf)

README.md – описание проекта и инструкция по запуску

requirements.txt, pyproject.toml – зависимости и настройки pytest

.gitignore – исключения для Git

Ссылка на репозиторий GitHub: https://github.com/schnchladp/stankin-lab